

Cursor v. Gemini CLI (Ping Pong Case Study)

For the Lab 1 Vibe Coding assignment, I used Cursor (Tool A) and Gemini CLI (Tool B) to generate a Ping Pong Table written in Python. This document analyzes and outlines each tools performances in the following criteria (as suggested by the Lab document): Code Quality, Generation Speed, User Interface, Debugging, and Customizability.

Code Quality:

When comparing both tools' approaches to writing code, each have shown to output two different approaches to coding.

Cursor wrote and stored all Ping Pong game functions ranging from player control, scorekeeping, paddle and ball behavior under a single class function. Moreover, all programs have all been written under a single .py file, including the ability to run the game.

Gemini CLI, however, takes a more modular approach to coding. Instead of a single class function, different aspects of the game such as paddle, scoreboard, or ball have their own unique classes and are packaged into different .py files. Then all of these files are imported and brought to use by game.py that ties all of these functions together.

In terms of comments and docstrings, both tools are not heavy on documentation, with Cursor being marginally better than Gemini CLI.

Comparing these two, in my opinion, Gemini CLI has a slightly better coding quality than Cursor. As Gemini CLI writes programs in a modular way, it's easier to understand how each aspect of the game is written, compared to the cluttered nature of Cursor. However, documentation, especially docstring is to be desired for both tools. Nonetheless these issues can easily be overcome by instruction both tools to be more intuitive of its program writing.

Generation Speed:

For generation speed, Cursor appeared to be faster in producing the code by double the speed in terms of personal experience compared to Gemini CLI. From ideation to generation, Cursor had only taken about few minutes (3 to 4 minutes). While Gemini took about a minute or more longer (note that the extra long time was also due to the interface design of Gemini CLI, which will be addressed in the next section).

User Interface:

Aside from coding quality, another area where both tools are distinctly different is in the interface design. For prompt handling, Cursor's is very similar to OpenAI's ChatGPT, in that it can handle multi-modal prompts and not just text alone. For example, the instructions given to Cursor agent on what qualifies as a Ping Pong game, as sent using a screenshot taken from the lab document. Outside of images, the prompt can receive voice recording or voice commands further enabling greater flexibility for users to send prompts. Additionally, Cursor agent offers users to change the model type, allowing them to tailor the model itself.

Gemini CLI is more restrictive than Cursor. As it uses a terminal interface, prompts can only be given through typed commands. Moreover, when dealing with typos, under the most extreme cases, users may have to delete previous words, sentences, or phrases to fix the issue instead and rewrite everything again.

However, both Cursor and Gemini CLI offer color coding to mark any changes it has made inside a program file. If both tools have added any new lines to the program, they would be highlighted as green/yellow, while lines that are deleted would be marked as red. This is arguably intuitive for engineers to easily observe the changes to the program before accepting the design change.

One area, however, Gemini CLI, in my opinion, does better than Cursor is the continuous request to the user to directly check its work and to accept it or not. Specifically, when generating the game, for each .py file, Gemini would display each change and would ask the user to confirm the code or suggest making any changes. This is a nice touch from a debugging perspective, as it allows users to provide immediate direct feedback - not just after all the code is written. However, at times this process of user consent or confirmation can increase the total work time.

When it comes to the learning curve, Gemini CLI, arguably has a much lower curve given its simplicity in design. As Cursor is an application that offers multiple tools, understanding and mastering them may take users some time.

Overall, though, the modern interface Cursor offers and its flexibility to handle different modes of prompts, can be an attractive tool over Gemini CLI, that offers greater potential.

Debugging:

As the program files made by Cursor and Gemini CLI, had little to no bug issues, there wasn't much need for debugging. However, prompts were given just in case for the model to investigate any potential bugs or issues the program had. In this instance, Cursor's functionality offers a more desirable action for the debugging process by changing the agent's settings. For Gemini, the debugging process was no different than previous workflow of asking the model to generate code.

Customization:

Finally, for customization, both tools are adequately good at accommodating additional requests to the program. For this lab, I asked both tools to modify the game into adding a single player mode, where a user would have to play against an AI. Both tools were able to accept and complete this request without any difficulties.